

Adding a New Vehicle of an Existing Type (Truck, Bus, Car)

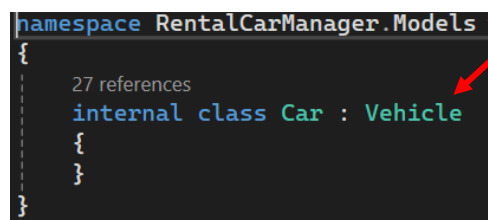
You can add new vehicles for any existing vehicle types directly within the program. Simply follow the prompts provided in the application to input the necessary vehicle details.

Extending the Vehicle Type List

If you want to add a new vehicle type (e.g., Motorcycle, Motorhome), follow these steps:

1. Create a New Model Class:

- Your new vehicle type should be stored in the Models folder and must inherit from the Vehicle base class. This ensures that it will be treated as a valid vehicle type in the system.

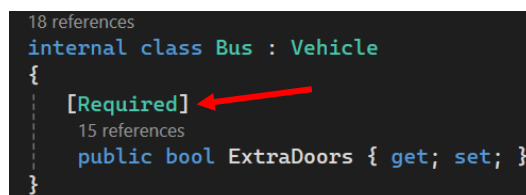


```
namespace RentalCarManager.Models
{
    27 references
    internal class Car : Vehicle
    {
    }
}
```

A screenshot of a code editor showing the definition of a new model class. The code is in C# and defines a namespace `RentalCarManager.Models`. Inside this namespace, there is a class `Car` that inherits from the `Vehicle` base class. The code is as follows: `namespace RentalCarManager.Models { 27 references internal class Car : Vehicle { } }`. A red arrow points to the `Vehicle` base class in the inheritance declaration.

2. Mark Required Properties (optional):

- Ensure that any required properties in your new model are marked with the `[Required]` data annotation. This ensures that these properties are validated and cannot be left empty.



```
18 references
internal class Bus : Vehicle
{
    [Required]
    15 references
    public bool ExtraDoors { get; set; }
}
```

A screenshot of a code editor showing the definition of a new model class. The code is in C# and defines a namespace `RentalCarManager.Models`. Inside this namespace, there is a class `Bus` that inherits from the `Vehicle` base class. The code is as follows: `18 references internal class Bus : Vehicle { [Required] 15 references public bool ExtraDoors { get; set; } }`. A red arrow points to the `[Required]` attribute applied to the `ExtraDoors` property.

3. Implement Optional Features:

- If your new vehicle type has optional features, ensure these features implement the `IOptionalFeatures` interface. This interface standardises how optional features are handled across different vehicle types. You must then declare this feature class as a property in either the vehicle base class or your new vehicle type model.

```
namespace RentalCarManager.Models
{
    8 references
    internal interface IOptionalFeatures
    {
        [NotMapped]
        5 references
        string GetDescription { get; }
    }
}
```

Note:

Properties and optional features defined in the new vehicle type class (as opposed to the base Vehicle class) can still be updated and filtered-by when renting a vehicle (provided they have the `[FilterName]` data annotation).

These properties and optional features can also be utilised when creating a new vehicle of that type, allowing you to populate the new vehicle with the specified features and properties defined in the vehicle type class.

However, please note that columns for these properties or features will not be visible in the vehicle overview table within the program ("View Vehicles" nor the "View currently active rentals" options).

4. Mark Filterable Properties:

- For any properties you want to be filterable in the UI, mark them with the `[FilterName]` data annotation. You can specify a display name within this attribute, which will be visible to users. This will allow users to filter vehicles based on these properties.

```
[FilterName("Daily rental rate (€)")]
55 references
public required double DailyRentalRate { get; set; }
```

5. System-Managed Properties:

- Properties that should not be modified by the user (such as CustomerInfo, which is automatically populated when a vehicle is rented) should be marked with the [SystemManaged] attribute. This designation ensures that these properties are modifiable only by the system, not by the user.

```
[SystemManaged]
6 references
public CustomerInfo? Customer { get; set; }
```

7. Update the OnModelCreating Method:

- Add the name of your new vehicle type to the OnModelCreating method in the VehicleContext class.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Vehicle>()
        .HasDiscriminator<string>("VehicleType")
        .HasValue<Vehicle>("Vehicle")
        .HasValue<Truck>("Truck")
        .HasValue<Bus>("Bus")
        .HasValue<Car>("Car");
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.Customer);
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.AC);
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.Radio);
}
```

- Make sure to include any optional features in the OnModelCreating method by using the appropriate configuration.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Vehicle>()
        .HasDiscriminator<string>("VehicleType")
        .HasValue<Vehicle>("Vehicle")
        .HasValue<Truck>("Truck")
        .HasValue<Bus>("Bus")
        .HasValue<Car>("Car");
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.Customer);
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.AC);
    modelBuilder.Entity<Vehicle>().OwnsOne(v => v.Radio);
}
```

Note:

Optional features declared in the new vehicle type class (instead of the base Vehicle class) should be specified using the new vehicle type class in the entity configuration. For example, use:

```
modelBuilder.Entity<NewVehicleType>().OwnsOne(v => v.NewFeature);
```

Instead of:

```
modelBuilder.Entity<Vehicle>().OwnsOne(v => v.Radio);
```

7. Include the New Vehicle Type in the Database_INITIALIZER:

- Ensure that you add at least one instance of your new vehicle type in the DbInitializer class within the Initialize method, and that all required properties are populated. This will seed the database with your new vehicle type and make it available for use within the program.

```
Truck[] trucks =  
[  
    new() { Make = "Volvo", Model = "FH", LicensePlate = "TRK321", Color = "Black",  
        Year = 2019, Acceleration = 12.0,  
        DailyRentalRate = 119.99, CargoCapacity = 20000.0,  
        Radio = new Radio{ MinFrequency = 12, MaxFrequency = 21} },  
];  
foreach (Truck t in trucks)  
{  
    context.Vehicles.Add(t);  
}  
context.SaveChanges();
```

8.Delete the Existing Database File:

- To ensure that the database is regenerated with the new models and changes, delete the existing database file. This will allow Entity Framework to create a new database with the updated schema when the application is run.